

ResearchAI

AI-Powered Research Paper Analysis Platform

Final Year Project — Viva Submission

Project	ResearchAI — Multi-Layer PDF Intelligence System
Domain	Artificial Intelligence / NLP / Full-Stack Web
Stack	Python · FastAPI · MySQL · Ollama LLM · ReportLab
Version	v3.1.0

Table of Contents

1. Project Overview & Objectives
2. System Architecture
3. System Requirements
4. Software Requirements
5. Project Module Descriptions
6. Database Schema
7. Full Step-by-Step Execution Guide
8. API Endpoint Reference
9. Pipeline Layer Summary
10. Troubleshooting & Common Errors
11. Conclusion

1. Project Overview & Objectives

What is ResearchAI?

ResearchAI is a full-stack, AI-powered web application designed to automate the analysis of academic research papers (PDFs). It accepts a PDF upload, runs it through a structured multi-layer processing pipeline, generates an AI reasoning report using a locally hosted Large Language Model (Ollama/DeepSeek), and exposes a streaming chat interface so users can ask questions about the paper in natural language.

Core Objectives

- ✓ Automate structured extraction of text, tables, figures, and key numbers from research PDFs.
- ✓ Audit extracted content for quality, completeness, and consistency with a scoring engine.
- ✓ Generate deep AI reasoning summaries (Abstract, Introduction, Methods, Results, Conclusion).
- ✓ Persist all results in a MySQL database and expose them through a secure REST API.
- ✓ Enable multi-turn, context-aware conversational chat about a processed paper via SSE streaming.
- ✓ Provide JWT-based user authentication with full register / login / logout support.

Key Innovation

Unlike generic chatbots, ResearchAI grounds every LLM response strictly in the extracted paper content. The LLM is given a rich `qa_context` (up to 14 000 characters) built from sections, tables, figures, and the reasoning summary — and is instructed to refuse answering anything not present in that context. This ensures hallucination-free academic Q&A.

2. System Architecture

Pipeline Overview

The system is structured as a sequential 5-layer pipeline. Each layer produces a well-defined output that feeds directly into the next layer. The entire pipeline runs asynchronously on the server — the client polls progress via Server-Sent Events (SSE).

L1	PDF Upload User uploads a PDF through the React frontend → FastAPI /api/upload endpoint saves it to temp_uploads/ and spawns an async pipeline task.
L2	Text Extraction text_extractor.py uses pdfplumber + PyMuPDF + Tesseract OCR to extract: sections (text, headings, page ranges), tables (headers + rows), figures (captions, semantic insights via CV), and key numbers with context.
L3	Audit & Validation audit_engine.py runs 6 check groups (A–F): structural completeness, section integrity, number consistency, cross-references, table integrity, and figure integrity. Produces a quality score (0–100) and grade (A–F).
L4	AI Reasoning reasoning_layer.py sends 6 targeted prompts to a local Ollama LLM (DeepSeek), summarising Abstract, Introduction, Methods, Results, Data, and Conclusion — then synthesises a final 5-section report.
L5	Output Formatting output_formatter.py merges all layer outputs into a single structured JSON, builds a qa_context string for chat, and saves everything to MySQL via job_outputs table.

Technology Stack

Component	Technology	Purpose
Backend API	FastAPI (Python)	REST endpoints, SSE streaming, async pipeline
LLM Engine	Ollama + DeepSeek v3.1:671b	Local AI reasoning and chat inference
PDF Processing	pdfplumber + PyMuPDF + Tesseract	Text, table, figure extraction + OCR
Database	MySQL 8 + PyMySQL	Users, jobs, sessions, messages
Authentication	JWT (python-jose + bcrypt)	Stateless auth, 72-hour tokens

Chat Streaming	SSE (text/event-stream)	Real-time token-by-token LLM output
Frontend	React (Vite build)	SPA served as static files by FastAPI
Image Processing	OpenCV + Pillow	Figure region detection and enhancement

3. System Requirements

Hardware Requirements

Component	Minimum	Recommended
CPU	4-core x86-64	8+ core (for Ollama inference)
RAM	8 GB	32 GB (LLM loads into RAM)
GPU (VRAM)	None (CPU mode)	NVIDIA 24 GB VRAM for full model
Disk Space	20 GB free	50 GB+ (LLM model ~40 GB)
OS	Windows 10 / Ubuntu 20.04	Ubuntu 22.04 LTS (recommended)
Network	Internet (setup only)	Localhost after model pull

Important Note on LLM

The DeepSeek v3.1:671b model requires significant RAM/VRAM. For a demonstration environment, a smaller model such as **llama3.2:3b** or **mistral:7b** can be substituted by setting the OLLAMA_MODEL environment variable accordingly.

Operating System Dependencies

- Tesseract OCR (sudo apt install tesseract-ocr or winget install UB-Mannheim.TesseractOCR)
- MySQL Server 8.0+ (sudo apt install mysql-server)
- Ollama runtime (curl -fsSL https://ollama.com/install.sh | sh)
- Python 3.10 or higher (python3 --version)
- Node.js 18+ (for building React frontend — npm run build)

4. Software Requirements (requirements.txt)

Python Package Dependencies

All Python dependencies are listed in requirements.txt at the project root. Install them in a virtual environment as shown in Section 7.

Package	Version	Purpose
fastapi	0.115.0	Async REST API framework
uvicorn[standard]	0.30.6	ASGI server for FastAPI
python-multipart	0.0.9	Multipart file upload handling
anthropic	0.34.0	Anthropic SDK (optional usage)
pdfplumber	0.11.4	PDF text and table extraction
pymupdf	1.24.10	PDF rendering and figure detection
pytesseract	0.3.13	OCR wrapper for Tesseract
pandas	2.2.2	Table data processing
opencv-python	4.10.0.84	Computer vision for figure extraction
Pillow	10.4.0	Image manipulation and enhancement
python-dotenv	1.0.1	Load .env configuration file
aiofiles	24.1.0	Async file I/O
pymysql	latest	MySQL database driver (add to req)
python-jose[cryptography]	latest	JWT token encoding/decoding
bcrypt	latest	Password hashing
requests	latest	HTTP calls to Ollama API

5. Project Module Descriptions

main.py

Role: FastAPI Application Entry Point

Key Functions / Classes:

- FastAPI app with CORS, static file serving, and SPA fallback
- `run_pipeline()` — async pipeline orchestrator (Layers 2–5)
- Auth routes: `/api/auth/register`, `/api/auth/login`, `/api/auth/me`
- Pipeline routes: `/api/upload`, `/api/status/{job_id}`, `/api/results/{job_id}`
- Session routes: CRUD for `chat_sessions`
- Chat route: `/api/chat` — streams LLM tokens via SSE
- Health check: `/api/health` — checks DB + Ollama connectivity

text_extractor.py

Role: Layer 2 — PDF Extraction Engine

Key Functions / Classes:

- `extract_paper(pdf_path)` — main entry point
- Uses `pdfplumber` for text/tables, `PyMuPDF` for figure bounding boxes
- Tesseract OCR for scanned figures with image enhancement
- `FigureSchema` dataclass — structured figure storage
- `CaptionExtractor` class — regex + zone-based caption detection
- Outputs: `sections[]`, `tables[]`, `figures[]`, `numbers[]`, `stats{}`

audit_engine.py

Role: Layer 3 — Audit & Validation Engine

Key Functions / Classes:

- `run_audit(doc)` — runs all 6 check groups, returns structured findings
- `check_structural_completeness()` — Group A (metadata, keys, sections)
- `check_section_integrity()` — Group B (page ranges, text length, ordering)
- `check_number_consistency()` — Group C (outliers, duplicates, % > 100)
- `check_cross_references()` — Group D (table/figure refs in text)
- `check_table_integrity()` — Group E (shape, headers, empty rows)
- `check_figure_integrity()` — Group F (captions, insights, page validity)
- `compute_score()` — CRITICAL=15pts, WARNING=5pts penalty, grade A–F

reasoning_layer.py

Role: Layer 4 — LLM Reasoning Engine

Key Functions / Classes:

- run_reasoning(unified_doc) — orchestrates 7 Ollama calls
- 6 chunk prompts: abstract, introduction, methods, results_text, results_data, conclusion
- 1 synthesis prompt: combines all chunks into 5-section final report
- check_ollama() — validates Ollama connectivity and model availability
- call_ollama() — HTTP POST to /v1/chat/completions with 600s timeout
- _fallback() — graceful fallback if synthesis fails

chat_engine.py

Role: Chat Streaming Engine

Key Functions / Classes:

- stream_chat(session_id, user_id, user_message) — sync SSE generator
- build_context(output) — builds 14k-char paper context for LLM
- _save_message() — persists user/assistant turns to messages table
- _touch_session() — updates chat_sessions.updated_at
- Maintains full conversation history per session turn

auth.py

Role: JWT Authentication Module

Key Functions / Classes:

- hash_password() / verify_password() — bcrypt with cost factor 12
- make_token() / _decode() — JWT HS256 with 72-hour expiry
- get_current_user() — FastAPI dependency (Bearer header)
- get_current_user_flexible() — supports ?token= query param for SSE
- register_user() / login_user() — DB-backed auth with validation

database.py

Role: MySQL Connection Factory

Key Functions / Classes:

- get_db() — returns a new PyMySQL DictCursor connection
- Reads config from .env: MYSQL_HOST, MYSQL_USER, MYSQL_PASSWORD, MYSQL_DB
- Caller is responsible for closing the connection

output_formatter.py

Role: Layer 5 — Output Formatter

Key Functions / Classes:

- format_output(doc, audit_result, reasoning_result) — merges all layers
- Produces: output_metadata, audit summary, reasoning report, document content
- Builds qa_context for the chat engine
- save_output() — writes output JSON to results/ directory

Auditor.py

Role: Layer 3 CLI Runner

Key Functions / Classes:

- Command-line interface for running audit on a JSON file directly
- Arguments: input_json, --output-dir, --show-passes, --strict
- print_terminal_summary() — coloured terminal output
- Saves audit JSON and Markdown audit report

report_generator.py

Role: Markdown Report Generator

Key Functions / Classes:

- generate_report(audit_result) — produces full Markdown audit report
- Includes score card, critical issues, warnings, info notes, passed checks
- Group breakdown table (A–F groups with counts)

Run_pipeline.py

Role: Standalone Pipeline Script

Key Functions / Classes:

- Runs the full pipeline (Layers 2–4) from command line without FastAPI
- Useful for testing individual PDFs during development
- PDF_PATH constant must be set at top of file

Schema.sql

Role: MySQL Database Schema

Key Functions / Classes:

- users — id, username, password_hash, display_name, last_seen, created_at
- job_outputs — job_id, user_id, filename, stem, output_json, output_path, report_path
- chat_sessions — id, user_id, job_id, name, filename, updated_at, created_at
- messages — id, session_id, role (user/assistant/system), content, created_at
- All foreign keys with CASCADE DELETE; indexes on user_id and updated_at

6. Database Schema

MySQL Tables (research_ai database)

Table: users

Column	Type	Constraint	Description
id	VARCHAR(36)	PK	UUID primary key
username	VARCHAR(80)	UNIQUE NOT NULL	Login username
password_hash	VARCHAR(255)	NOT NULL	bcrypt hashed password
display_name	VARCHAR(120)	NOT NULL DEFAULT ""	Display name
last_seen	DATETIME	NULL	Last login timestamp
created_at	DATETIME	NOT NULL DEFAULT NOW()	Registration time

Table: job_outputs

Column	Type	Constraint	Description
job_id	VARCHAR(36)	PK	UUID pipeline job ID
user_id	VARCHAR(36)	FK → users.id	Owner user
filename	VARCHAR(255)	NOT NULL	Original PDF filename
stem	VARCHAR(255)	NOT NULL	Filename stem (no extension)
output_json	LONGTEXT	NOT NULL	Full pipeline output as JSON
output_path	VARCHAR(512)	NOT NULL DEFAULT ""	Path on disk
report_path	VARCHAR(512)	NOT NULL DEFAULT ""	Audit report path
created_at	DATETIME	NOT NULL DEFAULT NOW()	Processing time

Table: chat_sessions

Column	Type	Constraint	Description
id	VARCHAR(36)	PK	UUID session ID
user_id	VARCHAR(36)	FK → users.id	Owner user
job_id	VARCHAR(36)	FK → job_outputs.job_id	Linked processed paper
name	VARCHAR(255)	NOT NULL DEFAULT 'New Chat'	Session name
filename	VARCHAR(255)	NOT NULL DEFAULT ''	Paper filename
updated_at	DATETIME	ON UPDATE CURRENT_TIMESTAMP	Last activity
created_at	DATETIME	NOT NULL DEFAULT NOW()	Creation time

Table: messages

Column	Type	Constraint	Description
id	VARCHAR(36)	PK	UUID message ID
session_id	VARCHAR(36)	FK → chat_sessions.id	Parent session
role	ENUM	user/assistant/system	Message sender
content	LONGTEXT	NOT NULL	Message body
created_at	DATETIME	NOT NULL DEFAULT NOW()	Message time

Run schema setup: `mysql -u root -p research_ai < Schema.sql`

7. Full Step-by-Step Execution Guide

Before You Begin

Follow every step in order. Do not skip any step. Steps marked with [WINDOWS] or [LINUX] are OS-specific. All other steps apply to both operating systems.

PHASE 1 — Environment Setup

1. Clone / Copy Project Files

Place all project files in a single folder, e.g. C:\projects\research_ai\ (Windows) or ~/projects/research_ai/ (Linux). Ensure the following files are present: main.py, auth.py, database.py, chat_engine.py, text_extractor.py, audit_engine.py, reasoning_layer.py, output_formatter.py, report_generator.py, Auditor.py, Run_pipeline.py, Schema.sql, requirements.txt

1. Create Python Virtual Environment

[Windows] python -m venv venv then venv\Scripts\activate [Linux] python3 -m venv venv then source venv/bin/activate You should see (venv) in your terminal prompt after activation.

1. Install Python Dependencies

With venv active, run: pip install -r requirements.txt Also install additional packages: pip install pymysql python-jose[cryptography] bcrypt requests Wait for all packages to install successfully. This may take 3–5 minutes.

1. Install Tesseract OCR

[Windows] Download installer from github.com/UB-Mannheim/tesseract/wiki → run .exe → install to C:\Program Files\Tesseract-OCR\ [Linux] sudo apt update && sudo apt install -y tesseract-ocr Verify: tesseract --version

1. Install Ollama and Pull LLM Model

[Windows] Download from ollama.com/download → run installer [Linux] curl -fsSL <https://ollama.com/install.sh> | sh After install, pull the model: ollama pull deepseek-v3.1:671b-cloud (For low-resource demo use: ollama pull mistral:7b then set OLLAMA_MODEL=mistral:7b in .env) Verify Ollama is running: ollama list

PHASE 2 — Database Setup

2. Install MySQL Server

[Windows] Download MySQL Installer from dev.mysql.com → install MySQL Server 8.0 → set root password during setup. [Linux] sudo apt install -y mysql-server → sudo mysql_secure_installation

2. 2	Create the Database Open MySQL client: <code>mysql -u root -p</code> Then run: <code>CREATE DATABASE research_ai CHARACTER SET utf8mb4 COLLATE utf8mb4_unicode_ci</code> ; Verify: <code>SHOW DATABASES</code> ; → you should see <code>research_ai</code> listed. Exit: <code>EXIT</code> ;
2. 3	Run the Schema SQL File [Windows] <code>mysql -u root -p research_ai < C:\projects\research_ai\Schema.sql</code> [Linux] <code>mysql -u root -p research_ai < ~/projects/research_ai/Schema.sql</code> This creates all 4 tables: <code>users</code> , <code>job_outputs</code> , <code>chat_sessions</code> , <code>messages</code> .
2. 4	Verify Tables Were Created <code>mysql -u root -p research_ai</code> Then: <code>SHOW TABLES</code> ; Expected output: <code>chat_sessions</code> , <code>job_outputs</code> , <code>messages</code> , <code>users</code> <code>EXIT</code> ;
2. 5	Test Database Connection Create a <code>.env</code> file (see Step 3.1), then run: <code>python test_db.py</code> Expected output: <code>SUCCESS</code> — MySQL connected!

PHASE 3 — Configuration (.env File)

3. 1	Create the .env File In your project root folder, create a new file named exactly <code>.env</code> (no other extension). Add the following contents (replace values as needed): <code>MYSQL_HOST=localhost</code> <code>MYSQL_PORT=3306</code> <code>MYSQL_USER=root</code> <code>MYSQL_PASSWORD=your_mysql_password_here</code> <code>MYSQL_DB=research_ai</code> <code>JWT_SECRET=replace_with_a_long_random_string_at_least_32_chars</code> <code>TOKEN_TTL_HOURS=72</code> <code>OLLAMA_BASE_URL=http://localhost:11434</code> <code>OLLAMA_MODEL=deepseek-v3.1:671b-cloud</code>
3. 2	Generate a Secure JWT Secret [Python] <code>python -c "import secrets; print(secrets.token_hex(32))"</code> Copy the output and paste it as the <code>JWT_SECRET</code> value in your <code>.env</code> file. Never commit the <code>.env</code> file to version control.

PHASE 4 — Starting the System

4. 1	Start Ollama Server Open a separate terminal/command prompt and run: <code>ollama serve</code> Leave this terminal open. Ollama must be running for any LLM functionality. Verify it is running: <code>curl http://localhost:11434/api/tags</code> (should return JSON)
4. 2	Activate Virtual Environment (if not already) [Windows] <code>venv\Scripts\activate</code> [Linux] <code>source venv/bin/activate</code>
4. 3	Start the FastAPI Backend Server From your project root folder, run: <code>uvicorn main:app --host 0.0.0.0 --port 8000 --reload</code> Expected output: <code>INFO: Started server process [...]</code> <code>INFO: Waiting for application startup.</code> <code>INFO: Application startup complete.</code> <code>INFO: Uvicorn running on http://0.0.0.0:8000</code>

4.4	Verify Backend Health Open your browser or use curl: <code>http://localhost:8000/api/health</code> Expected JSON response: <code>{ "status": "ok", "db_ok": true, "ollama_ok": true, ... }</code> If <code>db_ok</code> is false → check MySQL is running and <code>.env</code> credentials are correct. If <code>ollama_ok</code> is false → run <code>ollama serve</code> in a separate terminal.
4.5	Build and Serve the React Frontend (if applicable) If you have the React frontend source code: <code>cd frontend → npm install → npm run build</code> The build output goes to <code>frontend/dist/</code> . Copy its contents to the <code>static/</code> folder in your project root so FastAPI serves it automatically. Then open: <code>http://localhost:8000</code> in your browser.

PHASE 5 — Using the Application (API Workflow)

5.1	Register a User Account POST <code>http://localhost:8000/api/auth/register</code> Body (JSON): <code>{ "username": "testuser", "password": "test123", "display_name": "Test" }</code> Response: <code>{ "user_id": "...", "username": "testuser", "token": "eyJJ..." }</code> Save the token — you will need it for all authenticated requests.
5.2	Log In (Existing User) POST <code>http://localhost:8000/api/auth/login</code> Body (JSON): <code>{ "username": "testuser", "password": "test123" }</code> Response: same structure as register — includes a fresh JWT token.
5.3	Upload a Research PDF POST <code>http://localhost:8000/api/upload</code> Header: <code>Authorization: Bearer</code> Body: <code>multipart/form-data</code> , field name <code>'file'</code> , attach your <code>.pdf</code> file Response: <code>{ "job_id": "", "filename": "paper.pdf" }</code> Save the <code>job_id</code> — you will use it to track pipeline progress.
5.4	Monitor Pipeline Progress (SSE Stream) GET <code>http://localhost:8000/api/status/?token=</code> This is a Server-Sent Events stream. Each event looks like: <code>data: {"log": "Layer 2: Extracting paper...", "progress": 10, "status": "running"}</code> When status is <code>'done'</code> , the pipeline is complete. Progress goes: <code>5% → 10% → 25% → 40% → 60% → 80% → 100%</code>
5.5	Retrieve Pipeline Results GET <code>http://localhost:8000/api/results/</code> Header: <code>Authorization: Bearer</code> Returns the full output JSON containing: <code>audit score/grade</code> , <code>reasoning report</code> , <code>sections</code> , <code>tables</code> , <code>figures</code> , and <code>qa_context</code> .
5.6	Create a Chat Session POST <code>http://localhost:8000/api/sessions</code> Header: <code>Authorization: Bearer</code> Body: <code>{ "name": "My Paper Chat", "filename": "paper.pdf", "job_id": "" }</code> Response: <code>{ "session_id": "", "name": "My Paper Chat" }</code> Save the <code>session_id</code> .
5.7	Chat About the Paper (Streaming) POST <code>http://localhost:8000/api/chat</code> Header: <code>Authorization: Bearer</code> Body: <code>{ "message": "What is the main contribution of this paper?", "session_id": "" }</code> Response: SSE stream of tokens: <code>data: {"token": "The"} data: {"token": " main"} ... data: {"token": " __DONE__"}</code> Concatenate all tokens to build the complete response.

PHASE 6 — Running the Pipeline via Script (No Frontend)

6.
1

Edit Run_pipeline.py

Open Run_pipeline.py in a text editor. Find line: `PDF_PATH = r"C:\projects\research_ai\sample.pdf"`
Change the path to point to your actual PDF file. Save the file.

6.
2

Ensure Ollama is Running

In a separate terminal: `ollama serve` In another terminal, verify the model is available: `ollama list` You should see `deepseek-v3.1:671b-cloud` (or your chosen model) listed.

6.
3

Execute the Pipeline Script

With your virtual environment active, run: `python Run_pipeline.py` The terminal will display progress for each layer: Layer 2: Extraction → Layer 3: Audit → Layer 4: Reasoning (Ollama calls) This may take several minutes depending on your hardware.

6.
4

Check Output Files

After completion, the `results/` folder (auto-created) will contain: `_normalized.json` — bridged Layer 2 output `_audit.json` — Layer 3 audit findings and score `_audit_report.md` — Markdown audit report `data/reasoning/_reasoning.json` — Layer 4 output `data/reasoning/_reasoning.txt` — human-readable report

8. API Endpoint Reference

Authentication Endpoints

Method	Endpoint	Auth	Description
POST	/api/auth/register	None	Register new user, returns JWT token
POST	/api/auth/login	None	Login, returns JWT token
POST	/api/auth/logout	Bearer	Logout (stateless — client discards token)
GET	/api/auth/me	Bearer	Get current user info (id, username, display_name)

Pipeline Endpoints

Method	Endpoint	Auth	Description
POST	/api/upload	Bearer	Upload PDF, starts pipeline, returns job_id
GET	/api/status/{job_id}	?token=	SSE stream of pipeline progress logs
GET	/api/results/{job_id}	Bearer	Get full pipeline output JSON when done
GET	/api/recent-job	Bearer	Get most recent job_id for current user

Session & Chat Endpoints

Method	Endpoint	Auth	Description
POST	/api/sessions	Bearer	Create new chat session linked to a job
GET	/api/sessions	Bearer	List all sessions with message count
DELETE	/api/sessions/{session_id}	Bearer	Delete session and all its messages
PATCH	/api/sessions/{session_id}	Bearer	Rename a chat session
GET	/api/sessions/{session_id}/messages	Bearer	Get all messages in a session
POST	/api/chat	Bearer	Send message, get SSE token stream

Download & Health Endpoints

Method	Endpoint	Auth	Description
GET	/api/download/{job_id}/json	Bearer	Download output JSON file

GET	/api/download/{job_id}/report	Bearer	Download audit report Markdown file
GET	/api/health	None	System health (DB + Ollama status)

9. Pipeline Layer Summary

Layer	Name	Module	Input	Output	Score Gate
L1	Upload	main.py	PDF file	job_id, saved file	N/A
L2	Extraction	text_extractor.py	PDF path	sections, tables, figures, numbers	N/A
L3	Audit & Validation	audit_engine.py	Normalised doc JSON	findings, score 0–100, grade A–F	Score ≥ 75
L4	AI Reasoning	reasoning_layer.py	Normalised doc JSON	6 chunk summaries + final report	Ollama available
L5	Output Formatting	output_formatter.py	L2 + L3 + L4	output_json, qa_context	N/A
Chat	Q&A; Interface	chat_engine.py	session_id + message	SSE token stream	L5 complete

Audit Score Grading

Score Range	Grade	Status	Recommendation
90 – 100	A	EXCELLENT	Proceed to Layer 4 immediately
75 – 89	B	GOOD	Proceed to Layer 4
60 – 74	C	FAIR	Review warnings before proceeding
40 – 59	D	POOR	Fix critical issues before Layer 4
0 – 39	F	CRITICAL — DO NOT PROCEED	Re-extract PDF or check extractor

10. Troubleshooting & Common Errors

Error: `ModuleNotFoundError: No module named 'pymysql'`

Fix: Run: `pip install pymysql` (with `venv` activated). Also check: `pip install python-jose[cryptography] bcrypt requests`

Error: `FAILED: (2003, 'Can't connect to MySQL server')`

Fix: [Linux] `sudo service mysql start` or `sudo systemctl start mysql`
[Windows] Open Services → find MySQL80 → click Start
Then verify credentials in your `.env` file match your MySQL setup.

Error: `RuntimeError: Ollama is not reachable at http://localhost:11434`

Fix: Run `ollama serve` in a separate terminal and keep it running.
Verify with: `curl http://localhost:11434/api/tags`

Error: `RuntimeError: Model 'deepseek-v3.1:671b-cloud' not found in Ollama`

Fix: Run: `ollama pull deepseek-v3.1:671b-cloud`
Or use a smaller model: `ollama pull mistral:7b` then set `OLLAMA_MODEL=mistral:7b` in `.env`

Error: `401 Unauthorized on API calls`

Fix: Ensure you include the Authorization header: `Authorization: Bearer`
Tokens expire after 72 hours — log in again to get a fresh token.
For SSE endpoints, pass token as query param: `?token=`

Error: `PDF not found / extraction returns empty sections`

Fix: Check the PDF is not password-protected or image-only without OCR.
For scanned PDFs, ensure Tesseract is installed and in `PATH`.
Test: `tesseract --version` in terminal.

Error: Frontend shows blank page or 503

Fix: Run: `cd frontend && npm install && npm run build`
Copy build output (`dist/ contents`) into the `static/` folder in project root.
Restart `uvicorn` after copying.

Error: `test_db.py` shows `FAILED`

Fix: Double-check your `.env` values. Common issues:

- `MYSQL_PASSWORD` has wrong value
- `MYSQL_DB 'research_ai'` does not exist yet (run `CREATE DATABASE` first)
- MySQL server is not running

11. Conclusion

Project Summary

ResearchAI successfully demonstrates the integration of modern NLP, document processing, and full-stack web development into a cohesive, production-grade application. The project tackles the real-world challenge of extracting and understanding dense academic content by automating a pipeline that would otherwise require hours of manual effort.

Key Technical Achievements

- ★ Multi-modal PDF processing combining pdfplumber, PyMuPDF, OpenCV, and Tesseract OCR.
- ★ A 6-group automated audit engine with quantitative quality scoring (0–100) and grading.
- ★ Locally-hosted LLM integration (no cloud dependency) via Ollama with structured prompting.
- ★ JWT-based stateless authentication eliminating the need for session tables.
- ★ Asynchronous pipeline execution with real-time SSE progress streaming to the client.
- ★ Persistent multi-turn paper-grounded Q&A; chat with full conversation history per session.
- ★ Clean 5-layer architecture with well-defined interfaces between each processing stage.

Future Enhancements

- Support for batch PDF processing with queue management.
- Citation network analysis — extract and cross-reference the bibliography.
- Export reasoning reports as formatted PDF documents.
- Integration with reference managers (Zotero, Mendeley) via their APIs.
- Multi-language paper support using multilingual OCR and LLM models.